Министерство науки и высшего образования Российской Федерации
Пензенский государственный университет
Кафедра «Вычислительная техника»

ОТЧЕТ

по лабораторной работе №4

по дисциплине «Логика и основы алгоритмизации в инженерных задачах»

Тема: «Бинарное дерево поиска»

Выполнили: студенты гр. 25ВВВ1:
Димов Н.А.
Бардычев А.А.

Приняли:
к.т.н., доцент Юрова О. В.
ст. преподаватель Деев М. В.

Пенза 2026

1. Цель работы

Освоить структуру бинарного дерева поиска (БДП), научиться реализовывать основные операции: вставку, поиск, подсчёт вхождений, вставку без дубликатов. Также требуется оценить вычислительную сложность поиска в сбалансированном и вырожденном деревьях.

2. Лабораторное задание

Реализовать поиск заданного значения в уже построенном дереве.
Реализовать подсчёт числа вхождений указанного элемента.
*Изменить функцию добавления так, чтобы исключить повторную запись одинаковых значений.
*Оценить сложность процедуры поиска по значению в бинарном дереве.

3. Теоретические сведения

3.1. Определение и свойства бинарного дерева поиска (БДП)

Дерево — нелинейная связная иерархическая динамическая структура данных, состоящая из узлов (вершин) и направленных связей (ребер). Бинарным деревом называется дерево, у которого каждая вершина имеет не более двух непосредственных потомков, традиционно называемых левым и правым поддеревьями.

Бинарным деревом поиска (Binary Search Tree, BST) называется бинарное дерево, обладающее характеристическим свойством упорядоченности:

- для каждого узла X все ключи в его левом поддереве строго меньше ключа узла X (Key(left) < Key(X));
- все ключи в его правом поддереве больше либо равны ключу узла X (Key(right) >= Key(X));
- левое и правое поддеревья каждого узла также являются бинарными деревьями поиска.

Каждый узел бинарного дерева в динамической памяти представляется структурой, содержащей информационную часть и два адресных поля-указателя:

```
typedef struct Node {
   int data;
   struct Node *left;
   struct Node *right;
} Node;
```

3.2. Аналитическая оценка сложности алгоритмов в бинарных деревьях поиска

Временная сложность операций поиска, вставки и удаления в бинарном дереве поиска напрямую зависит от высоты дерева H (максимальной длины пути от корня до листа):

- Лучший случай: искомый элемент находится непосредственно в корне дерева. Требуется ровно 1 операция сравнения: $T_{best}(N) = O(1)$.
- Средний случай (сбалансированное дерево): при случайном порядке поступления ключей высота дерева составляет $H \approx \log_2 N$. При каждом шаге сравнения алгоритм отсекает примерно половину оставшихся узлов, обеспечивая логарифмическую сложность: $T_{avg}(N) = O(\log_2 N)$.
- Худший случай (вырожденное дерево): если элементы поступают в строго отсортированном порядке (по возрастанию или убыванию), бинарное дерево превращается в односвязный список, и глубина поиска возрастает до N. Сложность становится линейной: $T_{worst}(N) = O(N)$.

Рисунок 1 – Структурная организация бинарного дерева поиска: сбалансированное дерево со сложностью поиска $O(\log_2 N)$ и вырожденное дерево со сложностью $O(N)$.

4. Описание программной реализации

Программный комплекс разработан на языке Си. В программе реализованы следующие ключевые функции:

- createNode(int data) — выделяет память под новый узел с помощью malloc(), инициализирует информационное поле и обнуляет указатели left и right.
- insert(Node *tree, int data) — рекурсивно выполняет поиск позиции и вставку нового ключа в дерево поиска (с допущением дубликатов в правое поддерево).
- insertUnique(Node *tree, int data) — выполняет вставку ключа с контролем уникальности (**Задание 3***). Если ключ уже присутствует в дереве, операция отклоняется.
- search(Node *tree, int value) — выполняет поиск узла по ключу (**Задание 1**). При value < tree->data рекурсивно переходит влево, иначе вправо. Возвращает 1, если элемент найден, и 0 в противном случае.
- countOccurrences(Node *tree, int value) — рекурсивно подсчитывает количество вхождений заданного значения в дереве (**Задание 2**).
- printTree(Node *tree, int level) — наглядно отображает древовидную структуру на экране в текстовом виде с поворотом на 90 градусов (корень слева, правое поддерево вверху, левое — внизу) с пропорциональными

отступами.

- freeTree(Node *tree) — безопасно освобождает память всех узлов дерева, гарантируя отсутствие утечек оперативной памяти.
- main() — реализует интерактивное меню пользователя для работы с деревом.

Таблица 1 – Сравнительная характеристика вычислительной сложности операций в бинарном дереве поиска

Операция в БДП	**Лучший случай**	**Средний случай**	**Худший случай**	**Ограничивающий фактор / Условие**
Поиск ключа (search)	O(1)	$O(\log_2 N)$	O(N)	Высота дерева H; лучший: корень; худший: вырождение
Вставка ключа (insert)	O(1)	$O(\log_2 N)$	O(N)	Поиск позиции листа для размещения нового узла
Вставка без дубликатов (insertUnique)	O(1)	$O(\log_2 N)$	O(N)	Проверка наличия существующего узла перед вставкой
Подсчет вхождений (countOccurrences)	O(N)	O(N)	O(N)	Полный обход всех узлов дерева
Вывод дерева (printTree)	O(N)	O(N)	O(N)	Обход всех узлов для отображения
Освобождение памяти (freeTree)	O(N)	O(N)	O(N)	Рекурсивное освобождение памяти каждого узла

5. Выводы

- Изучены теоретические принципы организации динамических древовидных структур данных и особенности функционирования бинарных деревьев поиска (БДП).
- Разработана и программно реализована функция поиска элемента в дереве (search), показавшая логарифмическую зависимость числа сравнений от общего количества узлов.
- Реализован алгоритм подсчета количества вхождений заданного значения (countOccurrences), безошибочно определяющий кратность элементов в дереве с дубликатами.
- Успешно решена задача добавления узлов с исключением дубликатов (insertUnique), обеспечивающая строгое хранение множества уникальных

ключей и предотвращающая избыточный расход динамической памяти.

- Проведена аналитическая оценка сложности операций поиска, подтвердившая эффективность сбалансированного бинарного дерева поиска ($O(\log_2 N)$) по сравнению с линейным списком ($O(N)$).
- Разработанный программный комплекс на языке Си полностью выполнил все пункты лабораторного задания (включая задания повышенной сложности со звездочкой) и продемонстрировал надежное динамическое управление памятью без утечек.